Scrum Poker – Projektdokumentation

Modul	Web Technology (Leistungsnachweis «Scrum Poker App»)
Dozent	**TODO Gruppe: Name der Dozentin / des Dozenten eintragen**
Studiengang	**TODO Gruppe: Studiengang und Klasse eintragen**
Gruppenmitglieder	**TODO Gruppe: Vor- und Nachnamen aller Gruppenmitglieder eintragen**
Datum	11. Oktober 2026
Repository	https://github.com/gotazlib/abgaben/tree/main/web-technology/abgabe

Diese Dokumentation beschreibt, wie die App funktioniert, wie sie aufgebaut ist und warum sie so gebaut ist. Die Startanleitung in Kurzform steht im README. Alle Verweise der Form `Datei:Zeile` zeigen auf den Code im Projektordner (in der Abgabe der Ordner `code/` bzw. der Inhalt von `scrum-poker-code.zip`). Zur Vorbereitung aufs Fachgespräch liegt dort auch der Leitfaden docs/FACHGESPRAECH.md (auch als PDF und HTML): ein Rundgang durch alle Dateien, Musterantworten und Übungsaufgaben.

Inhalt

1. Funktionsweise
2. Architektur im Überblick
3. Komponentenstruktur und State
4. Routen und Navigation
5. REST-API
6. Datenfluss: Klick auf eine Karte
7. Lade- und Fehlerzustände
8. Strategie gegen Karteileichen
9. Durchschnitt bei T-Shirt-Grössen
10. Browser-Storage
11. Responsive Design und Box-Modell
12. Tests
13. Lokale Inbetriebnahme
14. Abgabe
15. KI-Einsatz

---

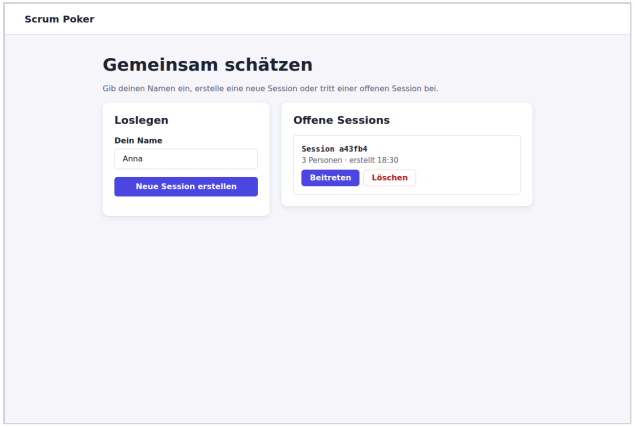
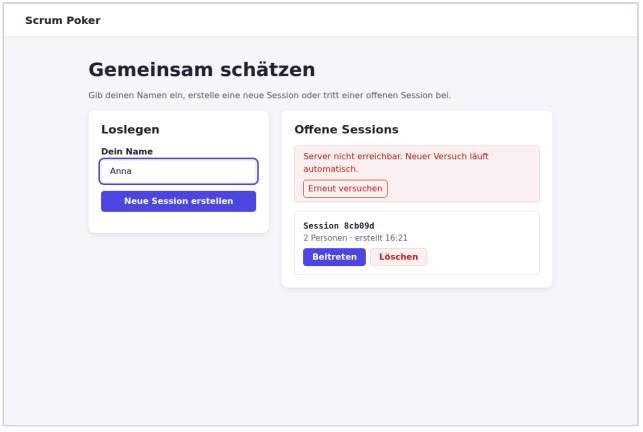
1. Funktionsweise

Scrum Poker (Planning Poker) ist eine Schätzmethode: Alle Teammitglieder legen gleichzeitig und verdeckt eine Karte mit ihrer Schätzung. Erst wenn alle gewählt haben, werden die Karten aufgedeckt. So beeinflusst niemand die anderen.

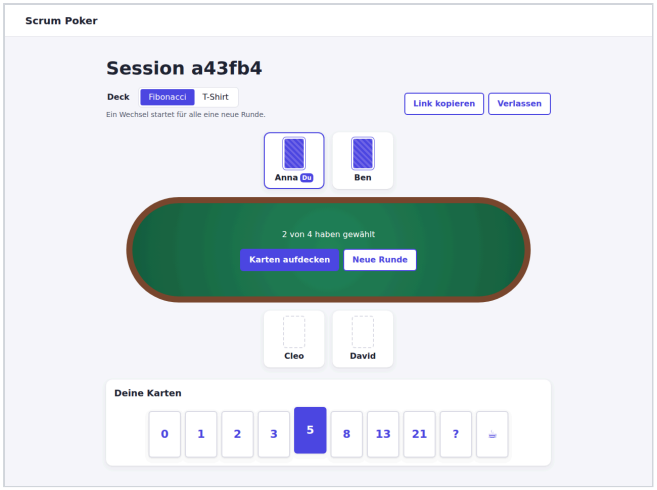
Ablauf in der App

1. **Name eingeben** (Home-Seite `/`). Der Name wird im Browser gespeichert und ist beim nächsten Besuch schon ausgefüllt. Ohne gültigen Namen (leer oder länger als 30 Zeichen) sind «Neue Session erstellen» und «Beitreten» gesperrt.
2. **Session erstellen oder beitreten.** Die Liste «Offene Sessions» aktualisiert sich alle 3 Sekunden von selbst und zeigt pro Session die Anzahl Personen und die Erstellungszeit. Sessions lassen sich nach einer Bestätigung löschen.
3. **Am Pokertisch** (`/sessions/:sessionId`) sitzen alle Teilnehmenden um einen Tisch. Unten liegen die eigenen Handkarten. Ein Klick auf eine Karte gibt die Stimme ab, ein zweiter Klick auf dieselbe Karte zieht sie zurück. Bei den anderen erscheint nur eine verdeckte Karte.
4. **Aufdecken:** von Hand mit «Karten aufdecken» oder automatisch, sobald alle gewählt haben. Danach werden Durchschnitt, Anzahl Stimmen und gegebenenfalls «Konsens!» angezeigt.
5. **Neue Runde** löscht alle Stimmen und verdeckt den Tisch wieder.
6. **Deck wechseln** zwischen Fibonacci (0, 1, 2, 3, 5, 8, 13, 21, ?, ☕) und T-Shirt-Grössen (XS, S, M, L, XL, ?). Ein Wechsel startet für alle eine neue Runde.
7. **Link kopieren** kopiert die Adresse der Session. Wer den Link öffnet und noch keinen Namen hat, wird zuerst nach dem Namen gefragt.
8. **Verlassen** führt zurück zur Home-Seite und entfernt die Person sofort aus der Session. Wer den Tab schliesst, wird nach spätestens 20 s entfernt (Kapitel 8). Ein **Reload** wirft niemanden raus: Man sitzt danach als dieselbe Person mit derselben Stimme am Tisch.

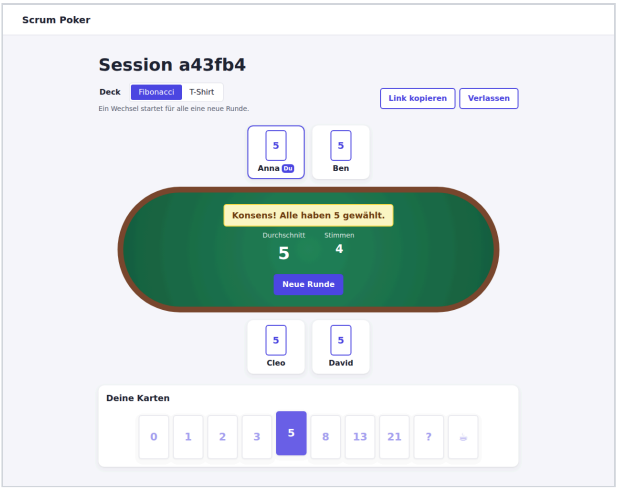
Alle Browserfenster sehen Änderungen der anderen spätestens nach 1.5 Sekunden (Polling).

Home-Seite (Desktop)	Server nicht erreichbar
	

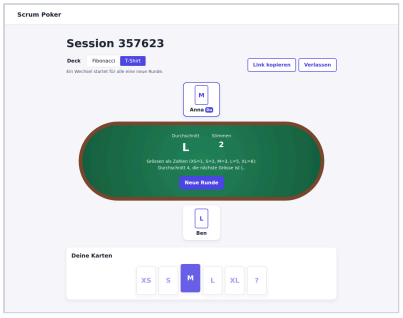
Pokertisch, verdeckt



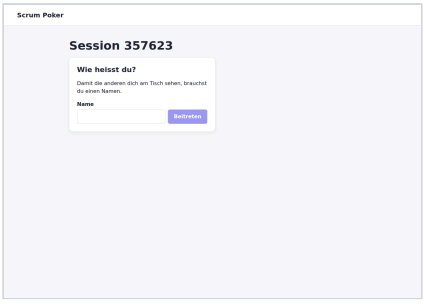
Nach dem Aufdecken mit Konsens



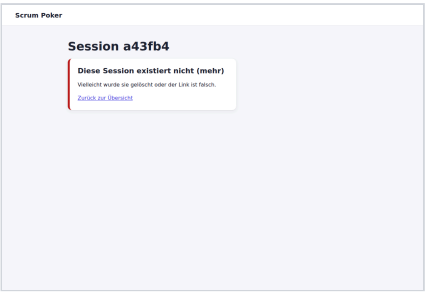
T-Shirt-Deck nach dem Aufdecken



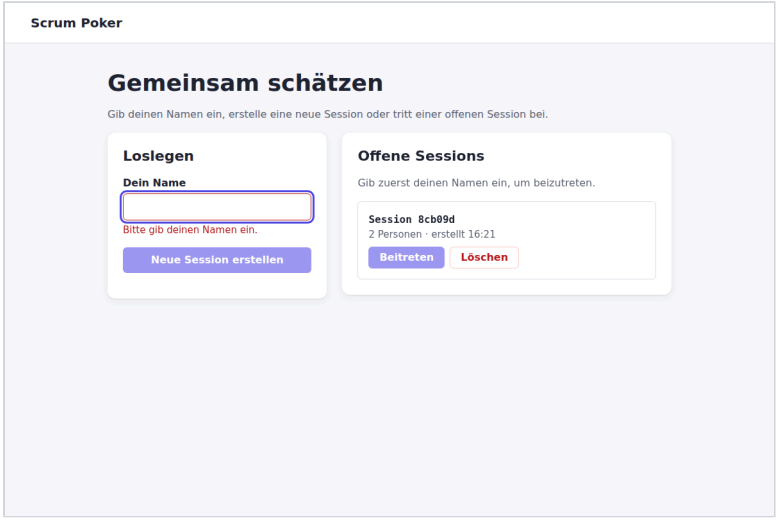
Name fehlt (geteilter Link)



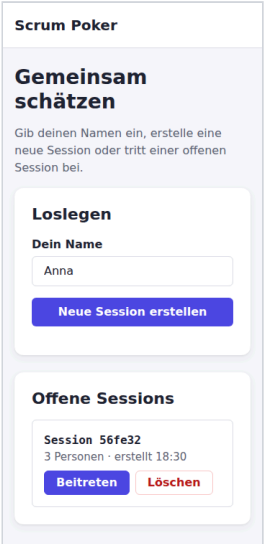
Session gelöscht



Ungültiger Name



Home mobil



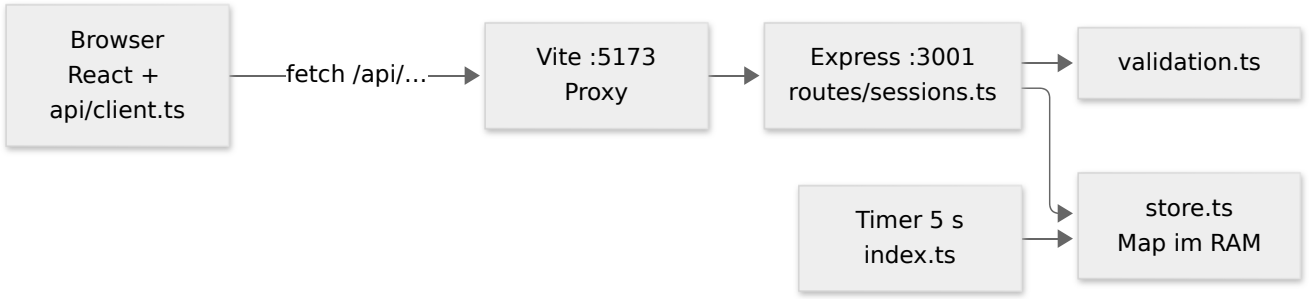
Pokertisch mobil



2. Architektur im Überblick

Das Projekt ist ein **pnpm-Workspace** mit drei Paketen (Dateibaum im README):

Paket	Technik	Aufgabe
`client` (`@scrum-poker/client`)	React 19, TypeScript, Vite, React Router 7	Oberfläche im Browser
`server` (`@scrum-poker/server`)	Node.js, Express 5, TypeScript (`tsx`)	REST-API, Daten im Arbeitsspeicher
`shared` (`@scrum-poker/shared`)	TypeScript	gemeinsame Types, Decks, `computeResult()`



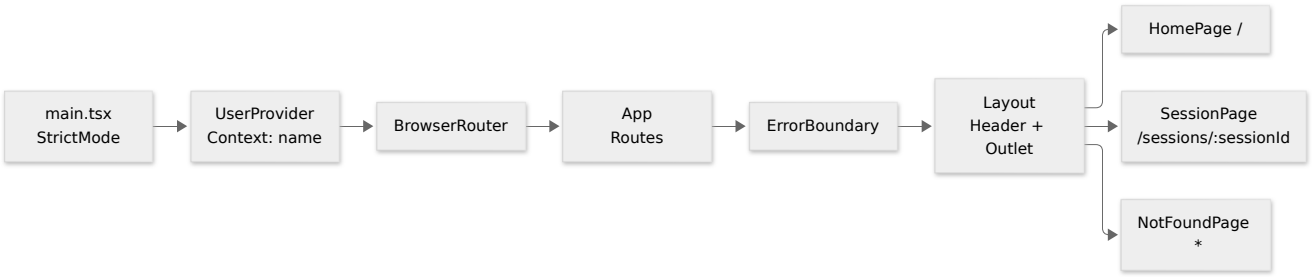
`shared/src/index.ts` (Types, `DECKS`, `computeResult`) wird von Client und Server importiert.

- **Ein Vertrag für beide Seiten:** Alle Request- und Response-Types stehen nur einmal in `shared/src/index.ts`. Ändert sich ein Feld, meldet TypeScript die Fehler im Client **und** im Server.
- **Kein CORS nötig:** Der Client ruft nur relative URLs wie `/api/sessions` auf. Der Vite-Dev-Server leitet sie an Port 3001 weiter (`client/vite.config.ts:18-20`). Für den Browser gibt es damit nur einen Ursprung (`localhost:5173`).
- **Schichten im Server:** Router (HTTP ⇄ Store, Statuscodes) → Validierung → Store (Daten und Regeln, ohne Express). Der Store lässt sich so ohne HTTP verstehen und testen.
- **Daten nur im Arbeitsspeicher:** Eine `Map<string, Session>` (`server/src/store.ts:26`). Nach einem Neustart des Servers sind alle Sessions weg. Für dieses Projekt ohne Deployment ist das ausreichend.

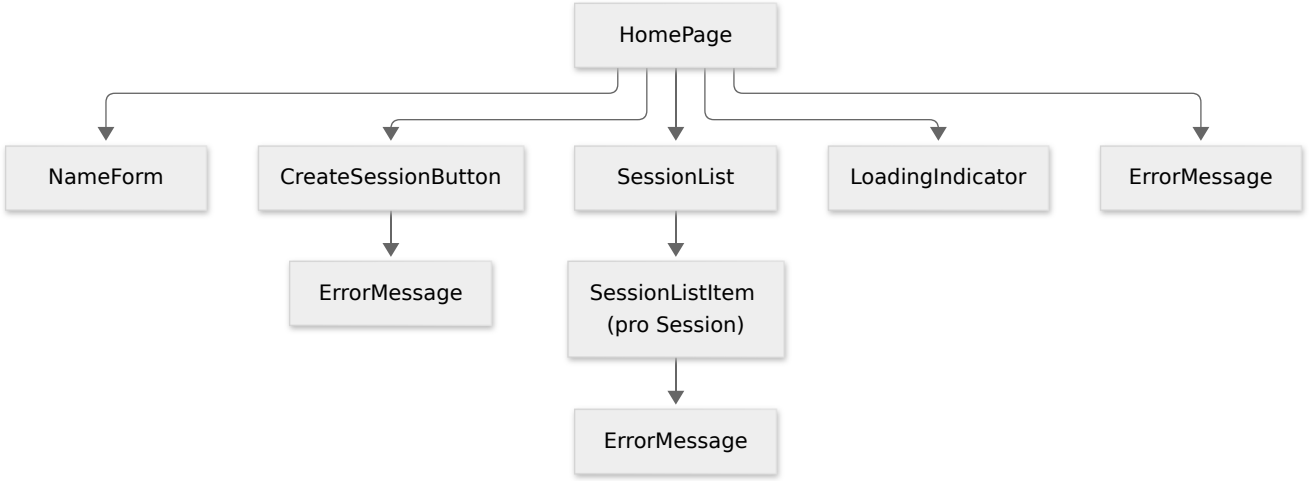
3. Komponentenstruktur und State

3.1 Komponentenbaum

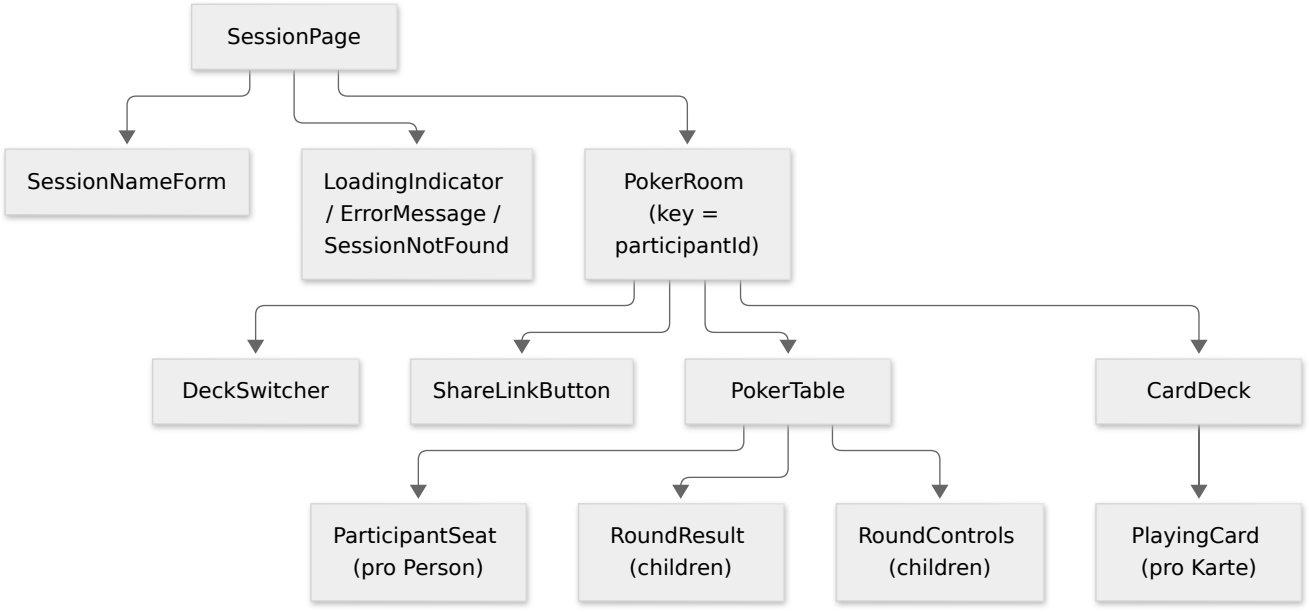
Gerüst und Routen



Home-Seite



Pokertisch



`RoundResult` und `RoundControls` werden von `PokerRoom` erzeugt und als `children` an `PokerTable` übergeben. `PokerTable` legt sie in die Tischmitte (Komposition statt vieler Props).

3.2 Komponenten, Props und wer sie rendert

Komponente	Gerendert von	Props	Eigener State
`App`	`main.tsx`	–	–
`ErrorBoundary` (Klasse)	`App`	`children`	`hasError`
`Layout`	Router (Eltern-Route)	– (rendert `<Outlet />`)	–
`HomePage`	Router (`/`)	–	– (liest Context, `usePolling`)
`NameForm`	`HomePage`	`name`, `error`, `onNameChange`	– (kontrolliert)
`CreateSessionButton`	`HomePage`	`disabled`	`isCreating`, `error`
`SessionList`	`HomePage`	`sessions`, `canJoin`, `onDeleted`	–

Komponente	Gerendert von	Props	Eigener State
`SessionListItem`	`SessionList`	`session`, `canJoin`, `onDeleted`	`isDeleting`, `error`
`SessionPage`	Router (`/sessions/:sessionId`)	– (`useParams`, `useUser`)	`participantId`, `joinError`, `joinAttempt`
`SessionNameForm`	`SessionPage`	– (schreibt in den Context)	`draft`
`PokerRoom`	`SessionPage`	`sessionId`, `participantId`, `onRejoin`	`pendingAction`, `actionError` + `usePolling`
`DeckSwitcher`	`PokerRoom`	`deck`, `disabled`, `onChange`	–
`ShareLinkButton`	`PokerRoom`	–	`copied`
`PokerTable`	`PokerRoom`	`participants`, `ownId`, `revealed`, `children`	–
`ParticipantSeat`	`PokerTable`	`participant`, `isOwn`, `revealed`	–
`RoundResult`	`PokerRoom` (in `PokerTable`)	`participants`, `deck`	– (berechnet)
`RoundControls`	`PokerRoom` (in `PokerTable`)	`revealed`, `canReveal`, `busy`, `revealing`, `resetting`, `onReveal`, `onReset`	–
`CardDeck`	`PokerRoom`	`deck`, `selected`, `disabled`, `onCardClick`	–
`PlayingCard`	`CardDeck`	`value`, `selected`, `disabled`, `onClick`	–
`LoadingIndicator`	`HomePage`, `SessionPage`, `PokerRoom`	`label?`	–
`ErrorMessage`	mehrere	`message`, `onRetry?`	–
`SessionNotFound`	`SessionPage`, `PokerRoom`	–	–
`NotFoundPage`	Router (`*`)	–	–

Alle Props sind mit einem `interface …Props` typisiert, z. B. `client/src/components/CardDeck.tsx:5-11`.

3.3 Wo liegt welcher State – und warum?

State	Ort	Begründung
Name	`UserContext` (`UserContext.tsx:10`), gespeichert in `localStorage`	Wird auf Home und am Pokertisch gebraucht, über Seiten hinweg. Darum liegt er über dem Router.
Gültigkeit des Namens	nirgends, berechnet mit `validateName(name)` (`HomePage.tsx:17`)	Abgeleiteter Wert. Gespeichert könnte er veralten.
Liste der Sessions	`usePolling` in `HomePage` (`HomePage.tsx:21`)	Nur die Home-Seite braucht sie, die Kinder bekommen sie über Props.

State	Ort	Begründung
`isCreating`, `isDeleting`, Aktionsfehler	in der Komponente mit dem Button	Betrifft nur diesen Button. Jeder Listeneintrag hat seinen eigenen Ladezustand.
`participantId`, `joinError`, `joinAttempt`	`SessionPage` (`SessionPage.tsx:41-44`)	Gehört zum Beitreten und Verlassen, das hier im `useEffect` passiert.
Session-Zustand (Teilnehmer, Deck, `revealed`, eigene Stimme)	`usePolling` in `PokerRoom` (`PokerRoom.tsx:35`)	**Single Source of Truth ist der Server.** Eigene Karte, Deck und «aufgedeckt» werden daraus gelesen, nicht doppelt gespeichert.
`pendingAction`, `actionError`	`PokerRoom` (`PokerRoom.tsx:39-40`)	Steuert den Ladezustand aller Aktions-Buttons gemeinsam.
Ergebnis, «x von y haben gewählt»	nirgends, bei jedem Render berechnet	Reine Ableitung aus dem Session-Zustand (`RoundResult.tsx:12-13`, `PokerRoom.tsx:80`).
`copied` («Kopiert!»)	`ShareLinkButton`	Betrifft nur diesen Button.
`draft` (Namensentwurf)	`SessionNameForm`	Erst beim Absenden wird der Name in den Context übernommen, sonst würde schon beim ersten Buchstaben beigetreten.

Single Source of Truth. Jede Information liegt an genau einer Stelle. Alles andere wird bei jedem Render daraus berechnet. Ein Beispiel: Die eigene Karte könnte man nach dem Klick in einem eigenen `useState` merken. Dann gäbe es aber zwei Wahrheiten. Nach «Neue Runde» durch eine andere Person bliebe die Karte lokal hervorgehoben, obwohl der Server die Stimme schon gelöscht hat. Darum liest `PokerRoom` die eigene Stimme aus der Server-Antwort (`me.vote`, `PokerRoom.tsx:64-78`). Der Server liefert dafür die eigene Stimme schon vor dem Aufdecken mit, nur die der anderen nicht (`server/src/store.ts:45`).

Lifting State Up. Alle Komponenten unter `PokerRoom` (`CardDeck`, `PlayingCard`, `DeckSwitcher`, `RoundControls`, `PokerTable`, `ParticipantSeat`, `RoundResult`) haben keinen eigenen State. Sie bekommen Daten über Props und melden Ereignisse über Callback-Props nach oben (`onCardClick`, `onChange`, `onReveal`, `onReset`). Der State liegt im nächsten gemeinsamen Vorfahren, der ihn braucht: `PokerRoom`. Gleich auf der Home-Seite: `NameForm` ist ein kontrolliertes Eingabefeld (`value={name}`, `onChange` → `onNameChange`), der Wert liegt im Context.

Context für den Namen. Den Namen braucht die Home-Seite (Eingabe) und die Session-Seite (Beitreten, Formular bei geteilten Links). Beide sind Geschwister unter dem Router. Ohne Context müsste `App` den Namen halten und über jede Route nach unten reichen. Mit Context liest jede Komponente ihn direkt mit `useUser()` (`client/src/context/useUser.ts`). `useUser()` wirft einen verständlichen Fehler, wenn kein `UserProvider` darüber liegt (`useUser.ts:6-8`).

Wann sind Props besser? Bei allem anderen in dieser App:

- Daten, die nur eine oder zwei Ebenen nach unten gehen (z. B. `sessions` an `SessionList`). Props machen sichtbar, woher die Daten kommen, und die Komponente bleibt ohne Provider wiederverwendbar und testbar.
- Häufig wechselnde Daten wie der Session-Zustand (alle 1.5 s). Ändert sich der Wert eines Contexts, rendern **alle** Komponenten neu, die ihn lesen. Bei Props rendert nur der betroffene Teilbaum.
- Darstellungs-Komponenten wie `PlayingCard`: Sie sollen nicht wissen, woher ihre Daten kommen.

Context lohnt sich für wenige, selten geänderte Werte, die viele Stellen auf verschiedenen Ebenen brauchen (Name, Sprache, Theme, eingeloggte Person).

`key={participantId}` auf `PokerRoom` (`SessionPage.tsx:109-117`): Nach «Wieder beitreten» gibt es eine neue `participantId`. Der neue `key` lässt React die alte `PokerRoom`-Instanz verwerfen und eine frische mit leerem State anlegen. So wird nie kurz der Zustand des alten Teilnehmers angezeigt.

3.4 Beitreten, Verlassen und React StrictMode

Beitreten und Verlassen passieren an genau einer Stelle: im Join-Effekt von `SessionPage` (`SessionPage.tsx:46-94`).

- **Beitreten** beim Mount mit `joinSession()`. Die Dependencies `[sessionId, trimmedName, hasName, joinAttempt]` sorgen dafür, dass bei einer anderen Session oder einem anderen Namen verlassen und neu beigetreten wird. «Erneut versuchen» und «Wieder beitreten» erhöhen nur `joinAttempt`.
- **Verlassen** im Cleanup. Der Button «Verlassen» navigiert nur auf `/`, dadurch wird die Seite entfernt und das Cleanup läuft. Es gibt also genau einen Leave-Request.
- **Reload:** Ein Reload baut React nicht ab, das Cleanup läuft also nicht, und man bleibt auf dem Server am Tisch. Nach dem Beitreten merkt sich der Effekt die `participantId` in `sessionStorage` (pro Tab). Die neu geladene Seite findet sie dort, prüft mit einem `GET`, ob der Server die Person noch kennt, und macht mit ihr weiter (`resumeOrJoin`). Platz und Stimme bleiben erhalten, es gibt keinen zweiten Eintrag, und die Runde wird nicht aufgedeckt. Kennt der Server die Person nicht mehr, wird normal neu beigetreten.
- **Tab schliessen:** Es wird kein Request gesendet. Der Heartbeat-Timeout entfernt die Person (Kapitel 8).
- **Ohne Namen** (geteilter Link in einem neuen Browser) wird nicht beigetreten, sondern `SessionNameForm` gezeigt. Speichert es den Namen im Context, läuft der Effekt erneut und tritt bei.
- **StrictMode** (in `main.tsx:9`) führt im Dev-Modus jeden Effekt beim ersten Mount zweimal aus: Mount → Cleanup → Mount. Dadurch starten zwei Joins. Jeder Effekt-Durchlauf hat sein eigenes Flag `cancelled` (`SessionPage.tsx:62`). Kommt die Antwort eines Joins erst, nachdem sein Effekt schon aufgeräumt wurde, verlässt dieser Teilnehmer die Session sofort wieder (`SessionPage.tsx:67-72`). Es bleibt genau ein Teilnehmer, ohne Duplikat und ohne verwaisten Eintrag. Nach einem Reload setzen beide Durchläufe mit der gespeicherten ID fort. Diese darf darum nicht verlassen werden, sonst wäre sie für den zweiten Durchlauf weg (`id !== savedId`). Zwei Tests prüfen das mit `<StrictMode>`.
- **Vom Server entfernt** (z. B. Laptop im Ruhemodus): Fehlt die eigene `participantId` in der Teilnehmerliste, zeigt `PokerRoom` «Du bist nicht mehr am Tisch» mit «Wieder beitreten».

---

4. Routen und Navigation

Die Routen stehen in `client/src/App.tsx:12-19`:

Pfad	Komponente	Zweck
`/`	`HomePage`	Name, Session erstellen, Liste offener Sessions
`/sessions/:sessionId`	`SessionPage`	Pokertisch einer Session
`*`	`NotFoundPage`	alle anderen Adressen, mit Link zurück

- Alle drei Routen sind Kinder einer Layout-Route ohne Pfad (`<Route element={<Layout />}>`). `Layout` zeigt den Header und rendert die aktive Seite an der Stelle von `<Outlet />`

(`Layout.tsx:11-13`).

- `BrowserRouter` umschliesst `App` in `main.tsx:11-13`. Er nutzt die History-API des Browsers: echte URLs ohne `#`, Vor- und Zurück-Button funktionieren.
- Der Header-Link ist ein `<Link to="/">`: Er wechselt die Seite ohne Neuladen.

Wie kommt die Session-ID in die Komponente? `:sessionId` ist ein URL-Parameter. `SessionPage` liest ihn mit `useParams()` aus (`SessionPage.tsx:33`):

```
const { sessionId = '' } = useParams()
```

`useParams()` liefert `string | undefined`. Der Standardwert `''` macht daraus einen `string`. `SessionPage` gibt die ID dann als Prop an `PokerRoom` weiter. Weil die ID in der URL steht, lässt sich jede Session per Link teilen und neu laden.

Programmatische Navigation mit `useNavigate()`:

Wo	Wann	Ziel
`CreateSessionButton.tsx:22`	nach erfolgreichem `POST /api/sessions`	`/sessions/<neue id>`
`SessionListItem.tsx:62`	Klick auf «Beitreten»	`/sessions/<id>`
`PokerRoom.tsx:101`	Klick auf «Verlassen»	`/`

«Beitreten» auf der Home-Seite navigiert nur. Das eigentliche Beitreten (`POST …/participants`) macht `SessionPage` beim Laden, denn dort landet man auch über einen geteilten Link.

---

5. REST-API

Basis-URL: `http://localhost:3001/api` (im Browser über den Proxy `http://localhost:5173/api`). Alle Bodies sind JSON. Jede Fehlerantwort hat die Form `{ "error": "…" }` (`ApiErrorBody`). Die Routen stehen in `server/src/routes/sessions.ts`, die Types in `shared/src/index.ts:67-112`.

Methode	Pfad	Body	Antwort	Statuscodes
`GET`	`/api/sessions`	–	`SessionSummary[]`	200
`POST`	`/api/sessions`	–	`{ id }`	201
`DELETE`	`/api/sessions/:sessionId`	–	–	204, 404
`GET`	`/api/sessions/:sessionId?participantId=…`	–	`SessionState`	200, 404
`POST`	`/api/sessions/:sessionId/participants`	`{ name }`	`{ participantId }`	201, 400, 404
`DELETE`	`/api/sessions/:sessionId/participants/:participantId`	–	–	204, 404
`PUT`	`/api/sessions/:sessionId/participants/:participantId/vote`	`{ value: CardValue | null }`	`SessionState`	200, 400, 404, 409
`PUT`	`/api/sessions/:sessionId/deck`	`{ deck: 'fibonacci' | 'tshirt' }`	`SessionState`	200, 400, 404
`POST`	`/api/sessions/:sessionId/reveal`	–	`SessionState`	200, 404
`POST`	`/api/sessions/:sessionId/reset`	–	`SessionState`	200, 404

Für alle Routen gilt zusätzlich: ungültiges JSON → 400, unbekannte Route → 404, unerwarteter Fehler → 500, immer als JSON und nie mit Stacktrace (`server/src/app.ts:16-31`).

Begründung der HTTP-Methoden

Methode	Verwendet für	Warum
`GET`	Liste, Session-Zustand	Liest nur, ist *sicher* und beliebig wiederholbar. Einzige Nebenwirkung: Mit `participantId` aktualisiert der Server `lastSeen` (Heartbeat). Das ist Verwaltungsinformation, keine fachliche Änderung.
`POST`	Session erstellen, beitreten	Erstellt eine neue Ressource, deren ID der **Server** vergibt → `201 Created`. Nicht idempotent: Zweimal senden erzeugt zwei Einträge.
`POST`	`reveal`, `reset`	Das sind Aktionen («aufdecken», «neue Runde»), keine Ressource mit einer vollständigen neuen Darstellung.
`PUT`	`vote`, `deck`	Der Client setzt einen Wert vollständig neu. **Idempotent:** Derselbe Request zweimal ergibt denselben Zustand. `value: null` zieht die Stimme zurück.
`DELETE`	Session, Teilnehmer	Entfernt eine Ressource → `204 No Content`. Teilnehmer entfernen ist idempotent: Auch ein schon entfernter Teilnehmer ergibt 204, denn das Ziel («nicht mehr dabei») ist erreicht.

Statuscodes

Code	Bedeutung in dieser API
200	OK, Antwort enthält Daten
201	Session oder Teilnehmer erstellt
204	Gelöscht, kein Body
400	Body ungültig: kein JSON, Name leer oder über 30 Zeichen, Karte nicht im aktiven Deck, unbekanntes Deck
404	Session unbekannt, Teilnehmer unbekannt (beim Abstimmen) oder Route unbekannt
409	Abstimmen, nachdem die Karten aufgedeckt sind (Konflikt mit dem aktuellen Zustand)
500	Unerwarteter Serverfehler

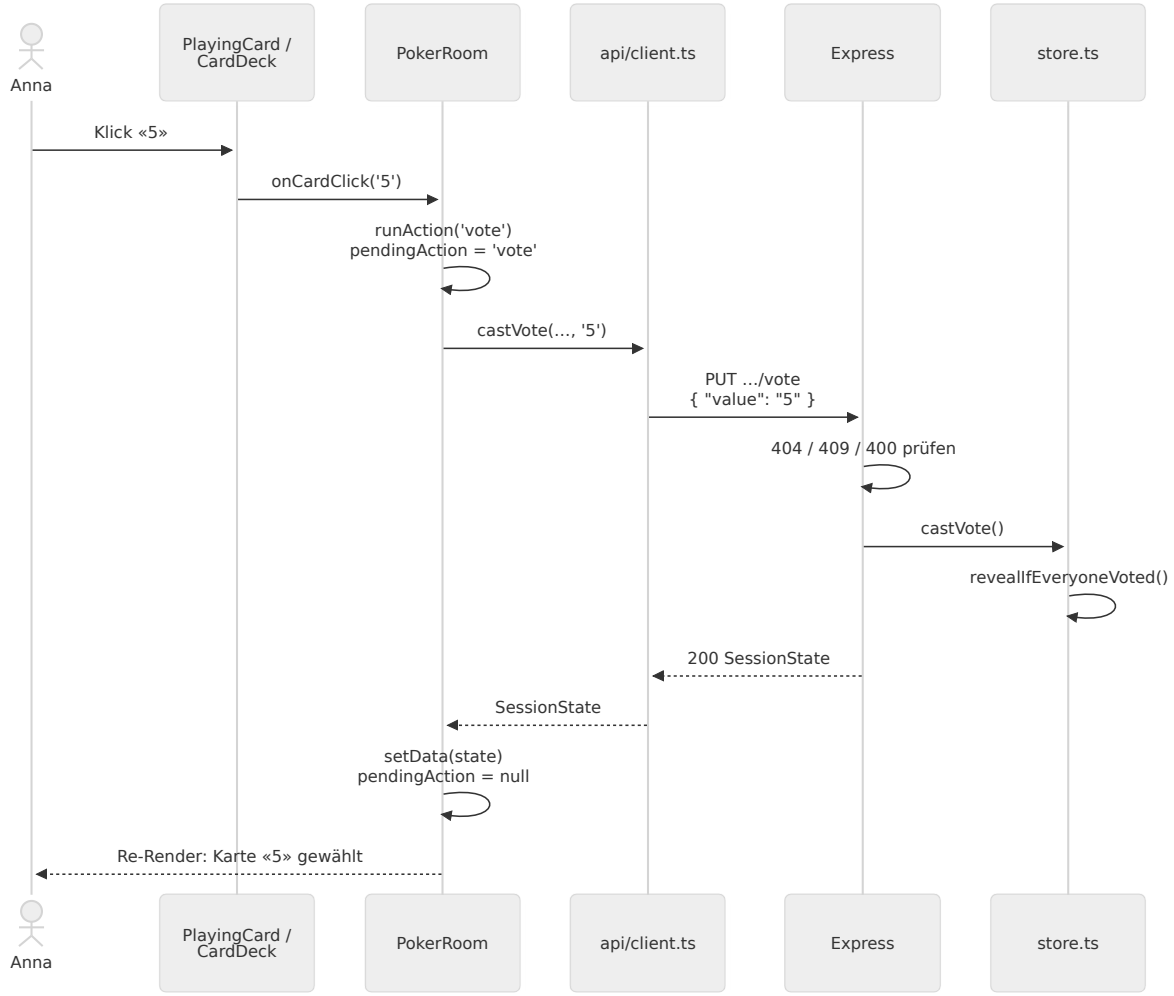
Fachliche Regeln im Server

- **Stimmen bleiben geheim:** Vor dem Aufdecken liefert der Server pro Person nur `hasVoted`, `vote` ist `null`. Ausnahme ist die eigene Stimme der fragenden Person (`store.ts:36-48`). Der echte Wert der anderen verlässt den Server also nie vorzeitig, auch nicht in den Entwicklertools.
- **Auto-Reveal:** Haben alle (mindestens eine Person) gewählt, deckt der Server auf. Das gilt auch, wenn die letzte noch fehlende Person geht (`store.ts:111-117`, `store.ts:150-157`).
- **Deck-Wechsel** startet eine neue Runde, weil alte Karten im neuen Deck ungültig wären. Dasselbe Deck nochmals wählen ändert nichts (`store.ts:170-177`).
- **Unbekannte `participantId` beim GET** ist kein Fehler (200). Der Client erkennt an der Teilnehmerliste selbst, dass er nicht mehr dabei ist, und bietet «Wieder beitreten» an.
- **Eingaben werden immer geprüft**, auch wenn der Client schon prüft. Ein Request kann auch von `curl` kommen. Der Body ist darum `unknown` und wird erst nach der Prüfung typisiert (`server/src/validation.ts`).

---

6. Datenfluss: Klick auf eine Karte

Beispiel: Anna klickt im Fibonacci-Deck auf «5». Ben sitzt am selben Tisch und hat noch nicht gewählt.



Schritt für Schritt am Code:

1. **Event.** `PlayingCard` ist ein echter `<button>` mit `onClick={onClick}` (`PlayingCard.tsx:15-23`). `CardDeck` hat ihm `onClick={() => onCardClick(value)}` mitgegeben (`CardDeck.tsx:26`). So erfährt `PokerRoom`, welche Karte geklickt wurde.
2. **Entscheidung.** `handleCardClick` vergleicht mit der eigenen Stimme aus dem Server-Zustand: Ist «5» schon gewählt, wird `null` gesendet (zurückziehen), sonst `'5'` (`PokerRoom.tsx:82-86`).
3. **State: Ladezustand.** `runAction` setzt `pendingAction = 'vote'` und löscht alte Fehler (`PokerRoom.tsx:44-54`). Dadurch ist `busy` `true`, alle Karten und Aktions-Buttons sind gesperrt (`PokerRoom.tsx:79`, `disabled={session.revealed || busy}` in `PokerRoom.tsx:132`). Doppelklicks senden so keinen zweiten Request.
4. **HTTP-Request.** `castVote()` baut den typisierten Body `CastVoteRequest` und schickt `PUT /api/sessions/:id/participants/:pid/vote` (`client.ts:128-135`). `send()` setzt den Header `Content-Type: application/json` und wandelt jeden Fehler in einen `ApiError` um (`client.ts:55-79`).
5. **Proxy.** Vite leitet `/api/…` an `localhost:3001` weiter (`vite.config.ts:18-20`).

6. **Server.** `express.json()` liest den Body (`app.ts:12`). Der Route-Handler (`sessions.ts:106-126`) prüft der Reihe nach: Session vorhanden (sonst 404), Teilnehmer vorhanden (sonst 404), noch nicht aufgedeckt (sonst 409), Karte im aktiven Deck (sonst 400, `validation.ts:38-50`).
7. **Store.** `castVote()` speichert die Stimme und ruft `revealIfEveryoneVoted()` auf (`store.ts:144-157`). Hätte Ben schon gewählt, wäre jetzt `revealed = true`.
8. **Antwort.** `sendSession(res, session, participant.id)` schickt den neuen `SessionState`. Annas eigener Eintrag enthält `vote: "5"`, alle anderen `vote: null` (`sessions.ts:125`, `store.ts:45`).
9. **State: neue Daten.** `runAction` übergibt die Antwort an `setData()` aus `usePolling` (`PokerRoom.tsx:48`). `setData` bricht eine noch laufende, ältere Polling-Anfrage ab, damit sie den alten Zustand nicht zurückbringt (`usePolling.ts:68-75`). `finally` setzt `pendingAction` zurück.
10. **UI.** React rendert `PokerRoom` neu. `myVote` ist jetzt `'5'`, `CardDeck` bekommt `selected="5"` und die Karte erhält `aria-pressed="true"` und die Klasse `playing-card--selected` (`PlayingCard.tsx:17-18`). `ParticipantSeat` zeigt an Annas Platz eine verdeckte Karte (`ParticipantSeat.tsx:22-27`). Der Text in der Tischmitte wird «1 von 2 haben gewählt».
11. **Andere Browser.** Ben sieht die verdeckte Karte bei seiner nächsten Polling-Abfrage (spätestens nach 1.5 s), weil sein `GET` jetzt `hasVoted: true` für Anna liefert.

---

7. Lade- und Fehlerzustände

Alle Requests laufen über `client/src/api/client.ts`. Jeder Fehler wird dort zu einem `ApiError` mit `status` und deutscher Meldung:

Ursache	`status`	Meldung	Stelle
Server läuft nicht, kein Netz (`fetch` wirft)	0	«Server nicht erreichbar.»	`client.ts:65-73`
Vite-Proxy erreicht Express nicht (502/503/504 ohne JSON)	502…	«Server nicht erreichbar.»	`client.ts:48-52`
Antwort mit `{ error }` (400, 404, 409, 500)	HTTP-Code	Text vom Server	`client.ts:39-53`
Request bewusst abgebrochen (`AbortController`)	–	wird unverändert weitergeworfen und ignoriert	`client.ts:66-70`

So behandelt jede Stelle der App die Anfragen:

Situation	Was die Benutzerin sieht	Code
Liste lädt das erste Mal	`LoadingIndicator` «Sessions werden geladen…»	`HomePage.tsx:45`
Server beim Polling der Liste nicht erreichbar	`ErrorMessage` mit «Erneut versuchen». Die letzte Liste bleibt sichtbar, das Polling versucht es weiter.	`HomePage.tsx:40-43`
Session erstellen fehlgeschlagen	Meldung unter dem Button, Button wieder aktiv	`CreateSessionButton.tsx:23-26`
Löschen fehlgeschlagen	Meldung im Listeneintrag. 404 gilt als Erfolg (schon gelöscht).	`SessionListItem.tsx:35-43`
Beitreten läuft	`LoadingIndicator` «Trete der Session bei …»	`SessionPage.tsx:106-108`

Situation	Was die Benutzerin sieht	Code
Unbekannte Session beim Beitreten (404)	`SessionNotFound` mit Link zur Übersicht	`SessionPage.tsx:100-102`
Anderer Fehler beim Beitreten	`ErrorMessage` mit «Erneut versuchen» (erhöht `joinAttempt`)	`SessionPage.tsx:103-105`
Session wird während des Pollings gelöscht (404)	`SessionNotFound`	`PokerRoom.tsx:56-58`
Erste Abfrage am Tisch läuft oder schlägt fehl	`LoadingIndicator` bzw. `ErrorMessage`	`PokerRoom.tsx:59-62`
Server antwortet beim Polling nicht	`ErrorMessage` «… Neuer Versuch läuft …». Der letzte Stand bleibt sichtbar.	`PokerRoom.tsx:106-107`
Vom Server entfernt (Timeout)	«Du bist nicht mehr am Tisch» + «Wieder beitreten»	`PokerRoom.tsx:64-76`
Aktion fehlgeschlagen (z. B. 409 nach dem Aufdecken)	`ErrorMessage`, die Oberfläche bleibt bedienbar	`PokerRoom.tsx:49-50`, `PokerRoom.tsx:108`
Aktion läuft	Alle Aktions-Buttons gesperrt. Aufdecken/Neue Runde mit Text «Wird aufgedeckt …» / «Wird zurückgesetzt …», bei Stimme und Deck-Wechsel werden Karten und Deck-Buttons ausgegraut (`aria-busy` am Tisch)	`RoundControls.tsx:26-38`, `PokerRoom.tsx:92-94`
Session erstellen / löschen läuft	Button gesperrt mit Text «Wird erstellt…» bzw. «Wird gelöscht…»	`CreateSessionButton.tsx:37`, `SessionListItem.tsx:73`
Verlassen	Die Seite wechselt sofort auf `/`, der Leave-Request läuft im Hintergrund. Schlägt er fehl, entfernt der Heartbeat-Timeout die Person, darum wird der Fehler bewusst ignoriert.	`SessionPage.tsx:85-90`
Ungültige Eingabe: Name leer, nur Leerzeichen oder zu lang	Meldung unter dem Feld, Buttons gesperrt (`aria-invalid`), auf der Home-Seite und im Namensformular eines geteilten Links	`NameForm.tsx:33-40`, `SessionNameForm.tsx:41-45`, `validateName.ts`
Ungültige Eingabe, die den Client umgeht (`curl`)	Server antwortet 400 mit `{ error }`	`validation.ts`
Unbekannte URL im Client	`NotFoundPage`	`App.tsx:17`
Fehler beim Rendern einer Komponente	`ErrorBoundary`: «Etwas ist schiefgelaufen» statt weisser Seite	`ErrorBoundary.tsx:17-32`

Keine unbehandelten Promises: Jede `async`-Funktion hat `try/catch` (`runAction`, `handleClick`, `handleDelete`), Promise-Ketten haben `.catch()` (Join/Leave in `SessionPage`). Event-Handler rufen `async`-Funktionen mit `void` auf, z. B. `onClick={() => void handleClick()}`. Das zeigt bewusst: Das Promise wird nicht weiter abgewartet, seine Fehler sind innen behandelt.

Die ErrorBoundary fängt nur Fehler **beim Rendern** ab, keine Fehler in Event-Handlern oder in `fetch`-Promises. Diese werden darum oben einzeln behandelt. React bietet Error Boundaries nur als Klassenkomponenten an (`static getDerivedStateFromError`).

Abbruch alter Requests: `usePolling` gibt jedem Request ein `AbortSignal`. Beim nächsten Poll, beim Unmount und bei `setData()` wird der laufende Request abgebrochen (`usePolling.ts:36-38`, `usePolling.ts:58-61`). So überschreibt eine langsame, alte Antwort nie neuere Daten, und nach dem Verlassen der Seite wird kein State mehr gesetzt.

---

8. Strategie gegen Karteileichen

Problem. Schliesst jemand den Tab, verliert das WLAN oder klappt den Laptop zu, bleibt die Person als «Karteileiche» in der Session. Sie stimmt nie ab, also löst auch das Auto-Reveal nie aus.

Die App arbeitet mit **zwei Schichten**: Wer den Tisch in der App verlässt, ist sofort weg. Wer einfach verschwindet (Tab zu, Absturz, Netz weg), wird vom Heartbeat-Timeout entfernt. Ein Reload gehört bewusst zu keiner der beiden: Er soll niemanden aus der Runde werfen.

Schicht 1: sofort verlassen (Button, Navigation in der App)

- «Verlassen» navigiert auf `/`. `SessionPage` wird entfernt, und das Cleanup des Join-Effekts sendet `DELETE …/participants/:id` (`SessionPage.tsx:83-93`). Dasselbe passiert bei jeder anderen Navigation weg vom Tisch (z. B. Zurück-Button, Header-Link).
- Weil nur das Cleanup verlässt, gibt es genau **einen** Leave-Request, egal auf welchem Weg man geht.

Schicht 2: Heartbeat über das Polling, 15 s Timeout, Aufräumen alle 5 s

Beim Schliessen des Tabs, wenn der Browser abstürzt, der Akku leer ist, das WLAN wegbricht oder der Laptop zugeklappt wird, läuft kein React-Cleanup und es wird nichts mehr gesendet. Dafür gibt es den Heartbeat:

1. Der Client fragt den Session-Zustand ohnehin alle **1.5 s** ab (`client/src/config.ts:4`) und schickt dabei seine `participantId` mit (`PokerRoom.tsx:30-35`).
2. Der Server wertet jede solche Abfrage als Lebenszeichen und setzt `lastSeen` auf jetzt (`sessions.ts:70-79`, `store.ts:120-122`).
3. Ein Timer läuft alle **5 s** (`server/src/index.ts:5`, `CLEANUP_INTERVAL_MS` in `server/src/config.ts:5`) und entfernt alle, deren `lastSeen` älter als **15 s** ist (`PARTICIPANT_TIMEOUT_MS`, `config.ts:9`; `store.ts:129-138`).
4. Das Entfernen läuft über `removeParticipant()`. Darum prüft es auch das Auto-Reveal: War die Karteileiche die Letzte ohne Stimme, wird jetzt aufgedeckt.
5. Bemerkt ein Client, dass er selbst nicht mehr in der Teilnehmerliste steht (z. B. nach dem Ruhemodus), bietet er «Wieder beitreten» an.

Warum diese Zahlen?

- 15 s sind zehn verpasste Abfragen. Ein kurzer Netzaussetzer oder ein langsamer Request wirft also niemanden raus.
- Mit 5 s Prüfintervall ist eine verschwundene Person nach spätestens 15 + 5 = **20 s** weg. Das ist kurz genug, dass das Team nicht lange wartet.
- Beide Werte sind benannte Konstanten in `config.ts`, nicht im Code verstreut.

Reload: dieselbe Person bleibt am Tisch

Ein Reload baut React ebenfalls nicht ab, die Person bleibt also auf dem Server am Tisch. Die neu geladene Seite macht mit derselben `participantId` weiter, die sie in `sessionStorage` gefunden

hat (Kapitel 3.4 und 10). Weil das Polling nach dem Reload sofort wieder läuft, greift der Timeout nicht. Platz und Stimme bleiben erhalten, und das Auto-Reveal löst nicht aus.

Verworfene Alternativen

Alternative	Warum verworfen
Nur ein Leave-Request beim Schliessen (ohne Heartbeat)	Unzuverlässig: Bei Absturz, Akku leer, Netzverlust oder Ruhemodus wird nie etwas gesendet.
Zusätzlich Leave beim `pagehide`-Event (Tab schliessen)	War in einer früheren Version eingebaut. `pagehide` feuert aber auch beim **Reload**, und der Browser kann die beiden Fälle nicht unterscheiden. Ein Reload hätte dann die Stimme gelöscht, und war die Person die Letzte ohne Stimme, hätte der Server für alle aufgedeckt. Darum verzichten wir darauf: Ein geschlossener Tab bleibt höchstens 20 s am Tisch, aufdecken von Hand geht in dieser Zeit trotzdem.
Eigener Heartbeat-Endpunkt (z. B. `POST …/heartbeat` alle 5 s)	Funktioniert, ist aber ein zusätzlicher Request und Code. Das Polling findet sowieso statt und erfüllt denselben Zweck.
Kürzerer Timeout (z. B. 5 s)	Ein Tab im Hintergrund drosselt seine Timer. Ein zu kurzer Timeout würde Personen entfernen, die nur kurz das Fenster gewechselt haben.
WebSockets: Verbindungsabbruch = weg	Erkennt Abbrüche sofort, braucht aber eine zweite Technik (WebSocket-Server, Reconnect-Logik). Der Kurs und die Aufgabe setzen auf REST und Polling.
Leere Sessions automatisch löschen	Laden alle kurz die Seite neu, wäre die Session sonst weg. Sessions werden nur explizit gelöscht.
`participantId` in `localStorage` statt `sessionStorage`	`localStorage` teilen sich alle Tabs. Zwei Tabs derselben Person wären dann ein einziger Teilnehmer, und Verlassen in einem Tab würfe auch den anderen raus.

---

9. Durchschnitt bei T-Shirt-Grössen

T-Shirt-Grössen haben keinen Zahlenwert. Gewählte Variante: **auf Fibonacci-ähnliche Zahlen abbilden, Durchschnitt rechnen, auf die nächste Grösse zurückrunden.**

Grösse	XS	S	M	L	XL
Wert	1	2	3	5	8

Beispiel: S (2) und L (5) ergeben 3.5 → am nächsten liegt M (3, Abstand 0.5) → **M**. Unter dem Ergebnis steht zur Nachvollziehbarkeit «Grössen als Zahlen (XS=1, …): Durchschnitt 3.5, die nächste Grösse ist M» (`RoundResult.tsx:33-38`).

Begründung

- Die Werte folgen der Fibonacci-Folge wie das Zahlen-Deck. Die Abstände wachsen mit der Grösse, so wie die Unsicherheit bei grossen Aufgaben. Mit 1–5 in gleichen Abständen hätte ein XL zu wenig Gewicht.
- **Gleichstand** (z. B. 4 genau zwischen M=3 und L=5) → die **grössere** Grösse. Beim Schätzen ist Aufrunden die vorsichtigere Wahl. Im Code ist das das `<=` in `shared/src/index.ts:143`.
- `?` und `☕` haben keinen Zahlenwert und zählen nicht zum Durchschnitt, aber zur Anzahl Stimmen. Gibt es gar keinen Zahlenwert, ist der Durchschnitt `null` und es wird «–» angezeigt.
- Der Durchschnitt wird auf eine Nachkommastelle gerundet (4.333 → 4.3).

- **Konsens** heisst: mindestens zwei Stimmen, und alle sind gleich (`index.ts:156`).

Verworfene Varianten: Den Median oder die häufigste Grösse anzeigen (verliert die Information, wie weit die Schätzungen auseinander liegen) oder gar keinen Durchschnitt (die Aufgabe verlangt einen).

Die Berechnung ist die reine Funktion `computeResult()` (`shared/src/index.ts:151-168`). Sie hängt weder von React noch von Express ab, könnte von Client und Server genutzt werden und ist ohne Browser testbar (`computeResult.test.ts`).

---

10. Browser-Storage

	Name	**Teilnehmer-ID**
Speicher	`localStorage` (bleibt nach dem Schliessen des Browsers erhalten, pro Ursprung `localhost:5173`)	`sessionStorage` (pro Tab, übersteht einen Reload, nicht das Schliessen des Tabs)
Key	`scrum-poker:name` (`UserContextValue.ts:7`)	`scrum-poker:participant:<sessionId>` (`SessionPage.tsx:14-16`)
Wert	der eingegebene Name als String	die `participantId` aus der Antwort auf das Beitreten
Zweck	Name ist beim nächsten Besuch schon ausgefüllt, ein geteilter Link braucht keine neue Eingabe	Nach einem Reload als dieselbe Person weitermachen (Kapitel 8)

Name (`localStorage`)

- **Lesen:** einmal beim ersten Render des `UserProvider`, als *Lazy Initial State*: `useState(() => localStorage.getItem(NAME_STORAGE_KEY) ?? '')` (`UserContext.tsx:10`). Die Funktion wird nur beim ersten Render ausgeführt, nicht bei jedem.
- **Schreiben:** in einem `useEffect`, der bei jeder Änderung von `name` läuft (`UserContext.tsx:12-14`). Der Effekt synchronisiert React-State mit einem System ausserhalb von React, genau dafür sind Effekte da. Jeder Tastendruck im Namensfeld ist damit sofort gespeichert, auch nach der Rückkehr vom Tisch.
- Der Präfix `scrum-poker:` verhindert Kollisionen mit anderen Apps auf `localhost`.
- Getestet in `UserContext.test.tsx`.

Teilnehmer-ID (`sessionStorage`)

- **Schreiben** nach erfolgreichem Beitreten, **Löschen** im Cleanup beim Verlassen (`SessionPage.tsx:73-75`, `SessionPage.tsx:85-90`).
- **Lesen** beim Start des Join-Effekts (`SessionPage.tsx:54-55`). Ob die gespeicherte ID noch gilt, entscheidet der Server: `resumeOrJoin()` fragt den Session-Zustand ab und tritt nur neu bei, wenn die Person dort fehlt (`SessionPage.tsx:20-30`).
- Getestet in `SessionPage.test.tsx` (Reload mit und ohne gültige ID, auch unter StrictMode) und im E2E-Test (Reload mitten in der Runde).

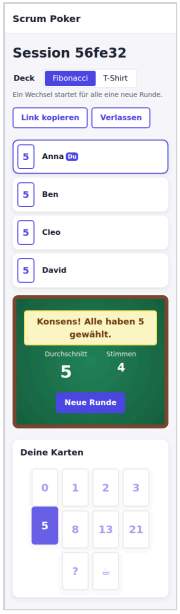
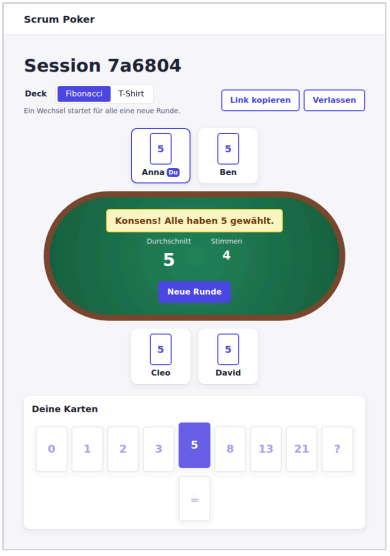
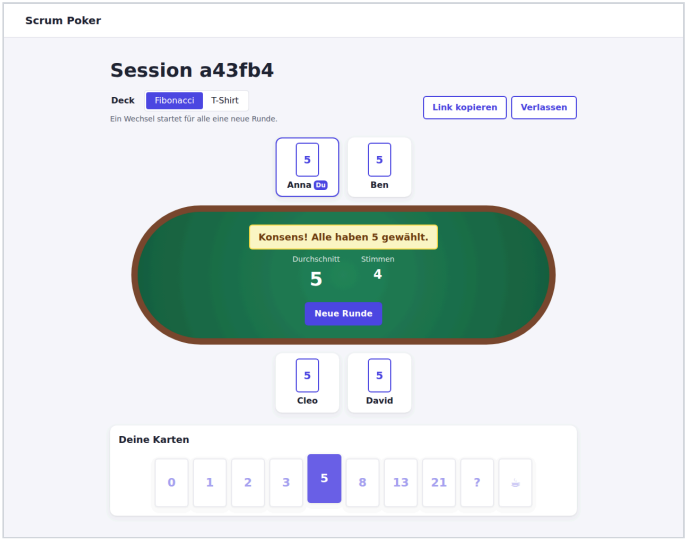
---

11. Responsive Design und Box-Modell

Mobile First. Die Grundregeln gelten für schmale Bildschirme, `@media (min-width: 768px)` ergänzt die Änderungen ab Tablet-Breite. Es gibt einen Breakpoint: **768 px**.

Bereich	Mobil (< 768 px)	Ab 768 px	Technik
Home-Seite	Karten untereinander	zwei Spalten 2 : 3	**CSS Grid**: `grid-template-columns: minmax(0, 2fr) minmax(0, 3fr)` (`HomePage.css:32-37`)
Pokertisch	alle Plätze als Liste, Tisch darunter (`order: 1`)	Plätze oben und unten, ovaler Tisch dazwischen	**Flexbox** `flex-direction: column` → `row` + `flex-wrap` (`PokerTable.css:43-66`)
Teilnehmerplatz	Karte links, Name rechts	Karte über dem Namen	Flexbox (`ParticipantSeat.css:94-109`)
Handkarten	brechen in mehrere Zeilen um	grössere Karten	Flexbox mit `flex-wrap: wrap` (`CardDeck.css`)
Inhalt	Rand `1.5rem 1rem`	Rand `2.5rem`, max. 960 px breit, zentriert	`global.css:105-123`

- Grid für das zweidimensionale Seitenlayout, Flexbox für Reihen von Elementen (Buttons, Karten, Plätze), die umbrechen dürfen.
- `minmax(0, …)` verhindert, dass ein langer Inhalt (z. B. ein langer Name) eine Grid-Spalte breiter als vorgesehen macht.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` (`client/index.html:6`) sorgt dafür, dass Handys die echte Breite verwenden und die Media Queries greifen.

Mobil (360 px)	Tablet (768 px)	Desktop (1280 px)
		

Box-Modell. Jedes Element besteht aus Inhalt, `padding`, `border` und `margin`. Global gilt `box-sizing: border-box` (`global.css:40-44`): `width` und `height` enthalten Padding und Rahmen. Ein Platz mit `width: 7.5rem` bleibt also 7.5rem breit, egal wie dick sein Rahmen ist. Mit dem Standard `content-box` würden Padding und Rahmen dazugerechnet, und Layouts würden überlaufen. Abstände kommen aus CSS-Variablen (`--space-xs` bis `--space-xl`, `global.css:20-25`), damit sie überall gleich sind.

Barrierefreiheit (Auswahl): echte `<button>`-Elemente, `aria-pressed` für gewählte Karte und aktives Deck, `role="alert"` für Fehler, `role="status"` für Ladeanzeigen, sichtbarer Fokusrahmen

(`:focus-visible`), versteckter Status-Text für Screenreader (`.visually-hidden`) und `prefers-reduced-motion` schaltet Animationen aus.

---

12. Tests

Alle Tests laufen mit **Vitest**. Im Client zusätzlich **React Testing Library** mit **jsdom**, im Server **supertest** (HTTP-Requests gegen die Express-App ohne echten Port).

```
pnpm test                              # alle Pakete
pnpm --filter @scrum-poker/server test  # nur ein Paket
pnpm --filter @scrum-poker/client exec vitest   # Client im Watch-Modus
```

Datei	Was getestet wird
`shared/src/computeResult.test.ts`	Durchschnitt Fibonacci, `?` / ☕ ignoriert, T-Shirt-Abbildung und Rückrundung, Gleichstand → grösser, kein Zahlenwert → `null`, Konsens erst ab zwei gleichen Stimmen
`server/src/app.test.ts`	alle Routen: erstellen/listen/löschen, 404 auf allen Routen, ungültiges JSON → 400, Namensprüfung (leer, Leerzeichen, 31 Zeichen, fehlt, keine Zahl), Verlassen idempotent, Stimmen geheim bis zum Aufdecken, eigene Stimme sichtbar, Auto-Reveal (auch beim Verlassen), Stimme zurückziehen, falsche Karte → 400, 409 nach dem Aufdecken, Reset, Deck-Wechsel, Heartbeat-Timeout mit Fake-Timern, leere Sessions bleiben
`client/src/App.test.tsx`	Routing: `/`, Session-ID aus der URL, Not-Found-Seite, Header auf jeder Seite
`client/src/pages/HomePage.test.tsx`	Name aus `localStorage`, sofort gespeichert, ungültiger Name sperrt Buttons, Session erstellen und navigieren, Fehler beim Erstellen, Ladeanzeige, Sortierung, leere Liste, Server weg, alte Liste bleibt, Beitreten, Löschen mit Bestätigung
`client/src/pages/SessionPage.test.tsx`	Join beim Mount und Leave beim Unmount, Reload setzt mit der `participantId` aus `sessionStorage` fort (auch unter StrictMode, ohne Join und ohne Leave), ungültige gespeicherte ID → neuer Join, Verlassen löscht die gespeicherte ID, StrictMode ohne Duplikat, ohne Namen kein Join (Meldung bei nur Leerzeichen), unbekannte und gelöschte Session, Stimme abgeben und zurückziehen, Aktionsfehler, verdeckte Karten, Durchschnitt und Konsens, `?` / ☕, Deck-Wechsel, Verlassen-Button, «Wieder beitreten», Server nicht erreichbar, «Kopiert!»
`client/src/hooks/usePolling.test.ts`	sofort laden und im Intervall, Fehler, `refresh()`, Abbruch beim Unmount, `setData()` verwirft ältere Anfrage
`client/src/api/client.test.ts`	URL und JSON-Body, 204 ohne Body, Fehlermeldung vom Server, Netzwerkfehler und 502 → «Server nicht erreichbar.», Abbruch wird weitergereicht
`client/src/context/UserContext.test.tsx`	Lesen und Schreiben von `localStorage`, Fehler ausserhalb des Providers
`client/src/utils/validateName.test.ts`	gültiger Name, leer, nur Leerzeichen, Länge ohne Rand-Leerzeichen

Datei	Was getestet wird
`e2e/scrum-poker.spec.ts`	**End-to-End mit Playwright** im echten Chromium gegen den echten Server: zwei Browser schätzen gemeinsam (erstellen, geteilter Link, verdeckt abstimmen inkl. Prüfung, dass die Server-Antwort die fremde Stimme nicht enthält, Auto-Reveal, Konsens, Deck-Wechsel, Reload mitten in der Runde ohne Duplikat und ohne Aufdecken, Verlassen, Zurück-Button, Löschen), Tab schliessen entfernt nach dem Heartbeat-Timeout, Fehlerzustände (ungültiger Name, unbekannte Session/URL, Server weg), kein horizontales Scrollen bei 360/768/1280 px. Jeder Test prüft, dass die Browser-Konsole keine Errors oder Warnings enthält.

Die E2E-Tests starten Server und Client selbst auf eigenen Ports (3101/5273, siehe `playwright.config.ts`) und laufen darum auch neben `pnpm dev`:

```
pnpm exec playwright install chromium   # einmalig (Linux: zusätzlich --with-deps)
pnpm test:e2e
```

Die Client-Tests ersetzen `fetch` durch einen kleinen Fake-Server (`SessionPage.test.tsx:10-77`). So testen sie das Verhalten der Oberfläche, ohne dass ein echter Server laufen muss.

Weitere Qualitätsprüfungen: `pnpm typecheck` (TypeScript `strict: true` in allen Paketen und in `e2e/`) und `pnpm lint` (ESLint mit `no-explicit-any` als Fehler, schlägt schon bei einer Warnung fehl).

---

13. Lokale Inbetriebnahme

Voraussetzungen

Werkzeug	Version	Prüfen mit
Node.js	**≥ 22.12** (aktuelle LTS empfohlen)	`node -v`
pnpm	10.x	`pnpm -v`
Browser	aktueller Chrome, Firefox, Edge oder Safari	–

Vite 8 und Vitest 5 laufen nicht mehr auf Node 20. Darum ist Node ≥ 22.12 Pflicht (steht auch in `package.json` unter `engines`).

Schritt für Schritt

1. **Node.js installieren** von https://nodejs.org (LTS) oder mit einem Versionsmanager (`nvm install --lts`). Danach `node -v` prüfen.
2. **pnpm aktivieren.** Node bringt Corepack mit, das pnpm bereitstellt:

   ```
   corepack enable
   pnpm -v
   ```

   Corepack liest `"packageManager": "pnpm@10.34.6"` aus `package.json` und verwendet genau diese Version (beim ersten Aufruf Download bestätigen). Ohne Corepack (z. B. Node 25+, wo es nicht mehr mitgeliefert wird) geht auch `npm install -g pnpm@10`.
3. **Projekt holen** – entweder das Code-ZIP entpacken oder das Repository klonen – und in den Projektordner wechseln:

```
cd scrum-poker            # aus dem ZIP
# oder: cd abgaben/web-technology/abgabe/code
```

4. **Abhängigkeiten installieren** (einmalig):

```
pnpm install
```

5. **Server und Client starten:**

```
pnpm dev
```

   Im Terminal erscheinen zwei farbige Präfixe: `[server]` meldet `Scrum Poker API running on http://localhost:3001`, `[client]` meldet die Vite-URL.

6. **Im Browser öffnen:** http://localhost:5173. Zum Ausprobieren ein zweites Fenster (oder ein privates Fenster mit anderem Namen) öffnen und derselben Session beitreten.

Dienst	URL	Port
Client (Vite-Dev-Server)	http://localhost:5173	5173
API (Express)	http://localhost:3001/api/sessions	3001
API über den Proxy	http://localhost:5173/api/sessions	5173

Weitere Befehle: `pnpm test`, `pnpm lint`, `pnpm typecheck`, `pnpm build` (siehe README). Beenden mit `Ctrl+C` im Terminal.

Troubleshooting

Problem	Ursache und Lösung
`Port 5173 is already in use`	Ein anderer Vite-Server läuft noch (`strictPort: true` verhindert ein stilles Ausweichen auf 5174, denn dann stimmte die URL nicht mehr). Alten Prozess beenden: `lsof -i :5173` (macOS/Linux) bzw. `netstat -ano | findstr :5173` (Windows) und den Prozess schliessen. Oder andere Ports wählen, siehe unten.
`EADDRINUSE: address already in use :::3001`	Ein alter Server läuft noch. Wie oben mit Port 3001.
Ports dauerhaft belegt	Ports per Umgebungsvariable ändern: `PORT=3002 API_PORT=3002 CLIENT_PORT=5174 pnpm dev` (macOS/Linux; Windows PowerShell: `$env:PORT=3002; $env:API_PORT=3002; $env:CLIENT_PORT=5174; pnpm dev`). `PORT` gilt für Express, `API_PORT` sagt dem Vite-Proxy, wohin er weiterleiten soll.
Rote Meldung «Server nicht erreichbar.»	Der Express-Server läuft nicht oder ist abgestürzt. Im Terminal nach `[server]`-Fehlern schauen, `pnpm dev` neu starten.
`Unsupported engine` / Syntaxfehler beim Start	Node-Version zu alt. `node -v` muss ≥ 22.12 sein.
`pnpm: command not found`	Schritt 2 (Corepack) ausführen oder Terminal neu öffnen.
`ERR_PNPM_…` bei `pnpm install`	Mit `npm` oder `yarn` installiert? `node_modules` löschen und nur `pnpm install` verwenden.
Alle Sessions weg	Der Server hat neu gestartet (z. B. nach einer Code-Änderung durch `tsx watch`). Die Daten liegen nur im Arbeitsspeicher, das ist so gewollt.

Problem	Ursache und Lösung
Eine Person bleibt nach dem Schliessen des Tabs oder einem Absturz am Tisch	Sie verschwindet nach höchstens 20 s (Heartbeat-Timeout, Kapitel 8).

---

14. Abgabe

Zwei gleichwertige Wege:

1. **ZIP-Datei:** Im Projektordner

   ```
   ./scripts/make-submission.sh
   ```

   ausführen. Es erzeugt `scrum-poker-code.zip` mit `git archive` aus dem committeten Stand: den Quellcode ohne `node_modules` und `dist`, dazu den Leitfaden fürs Fachgespräch (`docs/FACHGESPRAECH.md` / `.pdf` / `.html`). Diese Dokumentation (PDF und HTML) wird daneben abgegeben. Nach dem Entpacken laufen `pnpm install`, `pnpm test` und `pnpm build` ohne weitere Schritte. Das ZIP selbst ist nicht im Git (`.gitignore`).
2. **Git-Link** auf den öffentlichen Abgabe-Ordner (Dokumentation, Leitfaden, ZIP und Quellcode): https://github.com/gotazlib/abgaben/tree/main/web-technology/abgabe.

PDF und HTML dieser Dokumentation und des Leitfadens werden mit `pnpm docs:pdf` aus den Markdown-Dateien erzeugt (nach jeder Änderung erneut ausführen).

Bearbeitbare Version: `DOKUMENTATION.docx`. Nach dem Bearbeiten in Word Inhaltsverzeichnis aktualisieren und als PDF speichern. Die Word-Datei nutzt die HWZ-Vorlage und wird mit `pnpm docs:docx` aus dieser Markdown-Datei erzeugt. Das überschreibt Änderungen, die in Word gemacht wurden.

---

15. KI-Einsatz

Offenlegung

Die App wurde zum grössten Teil mit KI generiert. Code, Tests, CSS, Screenshots und diese Dokumentation hat ein KI-Agent geschrieben. Die Gruppe hat die Aufgaben formuliert, Anforderungen aus der Aufgabenstellung vorgegeben, Ergebnisse geprüft und die Änderungen freigegeben.

Werkzeug	Modell	Einsatz
Claude Code (CLI von Anthropic), mehrere Agenten mit je einem Pull Request pro Arbeitspaket	Claude Opus	gesamte Umsetzung, siehe unten

Arbeitspaket	Was die KI gemacht hat
Grundgerüst	pnpm-Workspace, Vite-/Express-Gerüst, TypeScript- und ESLint-Konfiguration, gemeinsame Types in `shared`
Server	REST-API, Store, Validierung, Heartbeat-Strategie, Server-Tests
Home-Seite	Komponenten, Context, `usePolling`, Löschen, Tests, Screenshots
Pokertisch	Join/Leave mit StrictMode, Karten, Decks, Aufdecken, Durchschnitt, Tests, Screenshots
CSS	Mobile-First-Layout, Tisch, Kartenanimation

Arbeitspaket	Was die KI gemacht hat
Abnahme	Prüfung gegen die Aufgabenstellung, Ports per Umgebungsvariable, E2E-Tests mit Playwright, Screenshots für Desktop/Tablet/Mobile
Dokumentation	diese Dokumentation, Leitfaden fürs Fachgespräch `docs/FACHGESPRAECH.md`, PDF-, HTML-, DOCX- und Abgabe-Scripts
Endkontrolle	Prüfung jeder Anforderung mit Checkliste; Reload setzt als dieselbe Person fort (`sessionStorage`), Namensprüfung auch beim geteilten Link sichtbar, Doku nachgeführt

Alle Arbeitsschritte sind in der Git-Historie nachvollziehbar.

Von Hand geschrieben oder angepasst

TODO Gruppe: Hier ehrlich auflisten, was ihr selbst geschrieben, geändert oder entschieden habt (Datei, Art der Änderung). Wenn nichts von Hand geschrieben wurde, das so stehen lassen und stattdessen beschreiben, was ihr geprüft und entschieden habt. Nichts erfinden.

Wie die Gruppe sichergestellt hat, dass sie den Code versteht

TODO Gruppe: Beschreiben, wie ihr den Code durchgearbeitet habt (z. B. Leitfaden docs/FACHGESPRAECH.md durchgearbeitet, Übungsaufgaben gelöst, Code gemeinsam besprochen).

Gruppenmitglieder und Aufteilung

TODO Gruppe: Namen aller Gruppenmitglieder und wer welchen Teil des Codes im Fachgespräch erklären kann.